

80-cereals

July 2, 2024

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
sns.set(color_codes=True)
```

```
[2]: #load the Dataset
df=pd.read_csv(r"C:\Users\dutta\Downloads\cereal.csv")
```

```
[3]: df
```

```
[3]:
```

	name	mfr	type	calories	protein	fat	sodium	fiber	\
0	100% Bran	N	C	70	4	1	130	10.0	
1	100% Natural Bran	Q	C	120	3	5	15	2.0	
2	All-Bran	K	C	70	4	1	260	9.0	
3	All-Bran with Extra Fiber	K	C	50	4	0	140	14.0	
4	Almond Delight	R	C	110	2	2	200	1.0	
..	...	..	...	...	...	...	...	...	
72	Triples	G	C	110	2	1	250	0.0	
73	Trix	G	C	110	1	1	140	0.0	
74	Wheat Chex	R	C	100	3	1	230	3.0	
75	Wheaties	G	C	100	3	1	200	3.0	
76	Wheaties Honey Gold	G	C	110	2	1	200	1.0	

	carbo	sugars	potass	vitamins	shelf	weight	cups	rating
0	5.0	6	280	25	3	1.0	0.33	68.402973
1	8.0	8	135	0	3	1.0	1.00	33.983679
2	7.0	5	320	25	3	1.0	0.33	59.425505
3	8.0	0	330	25	3	1.0	0.50	93.704912
4	14.0	8	-1	25	3	1.0	0.75	34.384843
..	...	...	...	...	...	...	...	...
72	21.0	3	60	25	3	1.0	0.75	39.106174
73	13.0	12	25	25	2	1.0	1.00	27.753301
74	17.0	3	115	25	1	1.0	0.67	49.787445
75	17.0	3	110	25	1	1.0	1.00	51.592193
76	16.0	8	60	25	1	1.0	0.75	36.187559

[77 rows x 16 columns]

```
[4]: # the first 5 rows of the Dataset
df.head()
```

```
[4]:
```

	name	mfr	type	calories	protein	fat	sodium	fiber	\
0	100% Bran	N	C	70	4	1	130	10.0	
1	100% Natural Bran	Q	C	120	3	5	15	2.0	
2	All-Bran	K	C	70	4	1	260	9.0	
3	All-Bran with Extra Fiber	K	C	50	4	0	140	14.0	
4	Almond Delight	R	C	110	2	2	200	1.0	

	carbo	sugars	potass	vitamins	shelf	weight	cups	rating
0	5.0	6	280	25	3	1.0	0.33	68.402973
1	8.0	8	135	0	3	1.0	1.00	33.983679
2	7.0	5	320	25	3	1.0	0.33	59.425505
3	8.0	0	330	25	3	1.0	0.50	93.704912
4	14.0	8	-1	25	3	1.0	0.75	34.384843

```
[5]: # the first 5 rows of the Dataset
df.tail()
```

```
[5]:
```

	name	mfr	type	calories	protein	fat	sodium	fiber	\
72	Triples	G	C	110	2	1	250	0.0	
73	Trix	G	C	110	1	1	140	0.0	
74	Wheat Chex	R	C	100	3	1	230	3.0	
75	Wheaties	G	C	100	3	1	200	3.0	
76	Wheaties Honey Gold	G	C	110	2	1	200	1.0	

	carbo	sugars	potass	vitamins	shelf	weight	cups	rating
72	21.0	3	60	25	3	1.0	0.75	39.106174
73	13.0	12	25	25	2	1.0	1.00	27.753301
74	17.0	3	115	25	1	1.0	0.67	49.787445
75	17.0	3	110	25	1	1.0	1.00	51.592193
76	16.0	8	60	25	1	1.0	0.75	36.187559

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 77 entries, 0 to 76
Data columns (total 16 columns):
#   Column      Non-Null Count  Dtype
---  -
0   name        77 non-null    object
1   mfr         77 non-null    object
2   type        77 non-null    object
```

```

3  calories  77 non-null    int64
4  protein   77 non-null    int64
5  fat       77 non-null    int64
6  sodium    77 non-null    int64
7  fiber     77 non-null    float64
8  carbo     77 non-null    float64
9  sugars    77 non-null    int64
10 potass    77 non-null    int64
11 vitamins  77 non-null    int64
12 shelf     77 non-null    int64
13 weight    77 non-null    float64
14 cups      77 non-null    float64
15 rating    77 non-null    float64
dtypes: float64(5), int64(8), object(3)
memory usage: 9.8+ KB

```

```
[7]: # Descriptive Statistics of the Dataset
df.describe()
```

```
[7]:
```

	calories	protein	fat	sodium	fiber	carbo	\
count	77.000000	77.000000	77.000000	77.000000	77.000000	77.000000	
mean	106.883117	2.545455	1.012987	159.675325	2.151948	14.597403	
std	19.484119	1.094790	1.006473	83.832295	2.383364	4.278956	
min	50.000000	1.000000	0.000000	0.000000	0.000000	-1.000000	
25%	100.000000	2.000000	0.000000	130.000000	1.000000	12.000000	
50%	110.000000	3.000000	1.000000	180.000000	2.000000	14.000000	
75%	110.000000	3.000000	2.000000	210.000000	3.000000	17.000000	
max	160.000000	6.000000	5.000000	320.000000	14.000000	23.000000	

	sugars	potass	vitamins	shelf	weight	cups	\
count	77.000000	77.000000	77.000000	77.000000	77.000000	77.000000	
mean	6.922078	96.077922	28.246753	2.207792	1.029610	0.821039	
std	4.444885	71.286813	22.342523	0.832524	0.150477	0.232716	
min	-1.000000	-1.000000	0.000000	1.000000	0.500000	0.250000	
25%	3.000000	40.000000	25.000000	1.000000	1.000000	0.670000	
50%	7.000000	90.000000	25.000000	2.000000	1.000000	0.750000	
75%	11.000000	120.000000	25.000000	3.000000	1.000000	1.000000	
max	15.000000	330.000000	100.000000	3.000000	1.500000	1.500000	

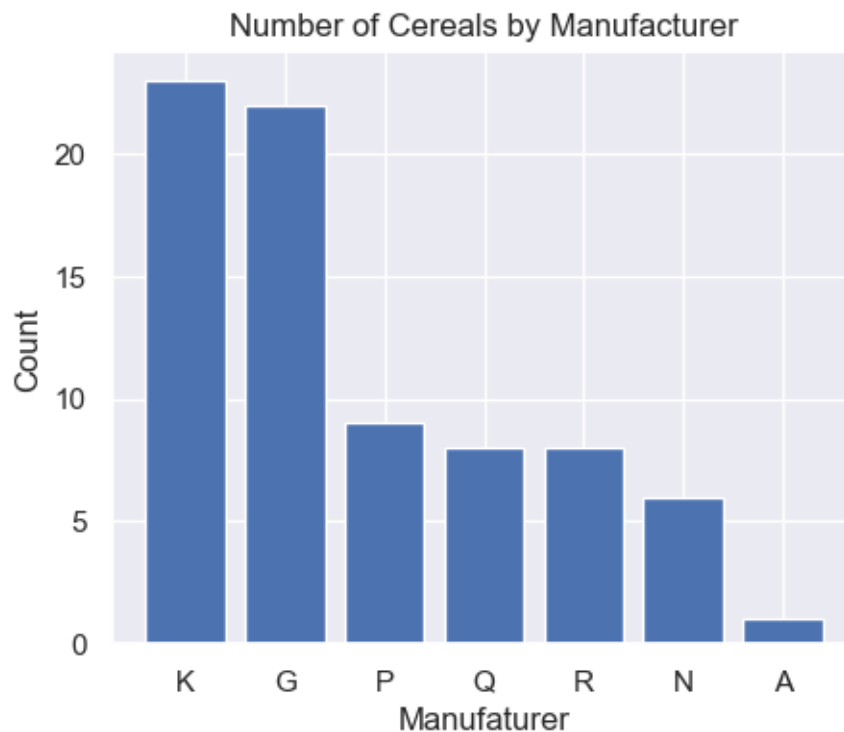
	rating
count	77.000000
mean	42.665705
std	14.047289
min	18.042851
25%	33.174094
50%	40.400208
75%	50.828392

max 93.704912

```
[8]: # check for duplicates
duplicates = df.duplicated()
```

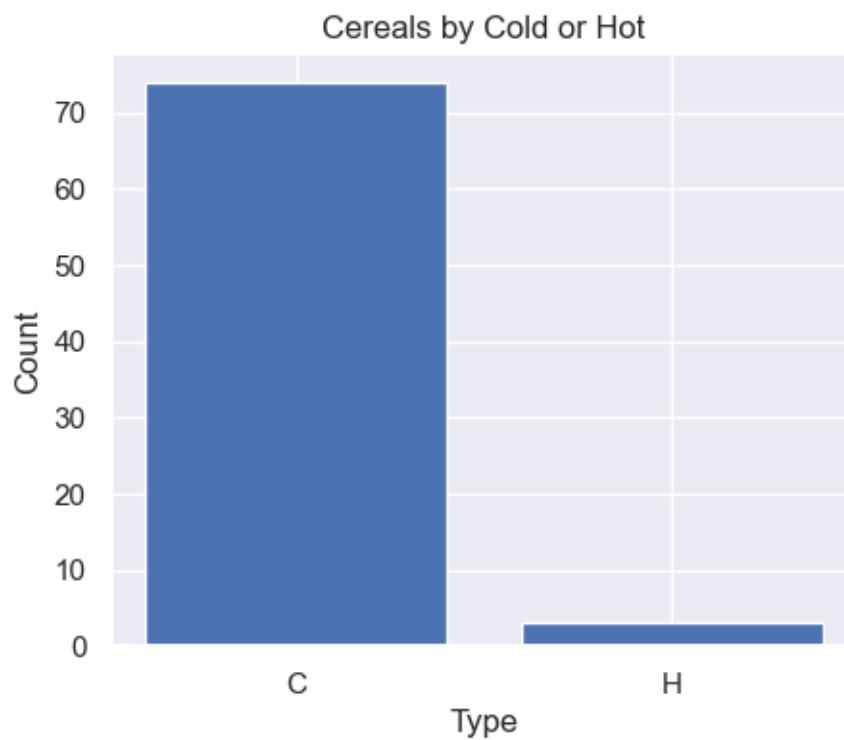
```
[9]: # checking for null values
null_counts= df.isnull().sum()
if null_counts.sum()>0:
    print(null_counts[null_counts>0])
```

```
[12]: # create a Bar plot
plt.figure(figsize=(5,4))
plt.bar(df['mfr'].value_counts().index, df['mfr'].value_counts().values)
plt.xlabel('Manufaturer')
plt.ylabel('Count')
plt.title('Number of Cereals by Manufacturer')
plt.show()
```

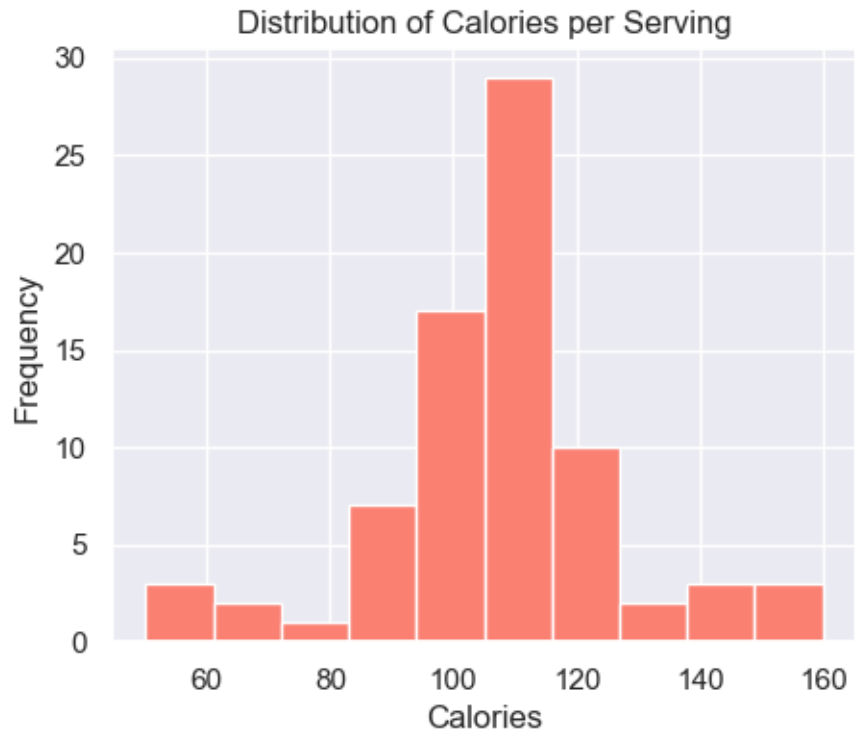


```
[14]: # create a Bar plot
plt.figure(figsize=(5,4))
plt.bar(df['type'].value_counts().index, df['type'].value_counts().values)
plt.xlabel('Type')
plt.ylabel('Count')
```

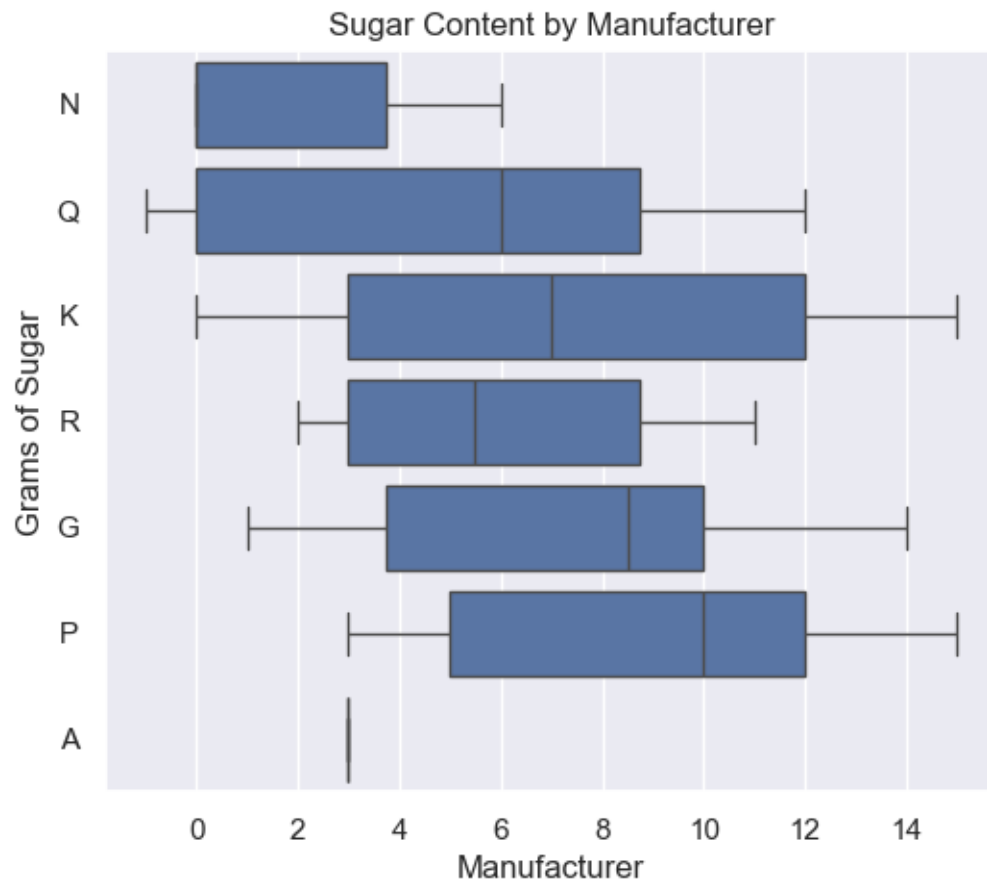
```
plt.title('Cereals by Cold or Hot')
plt.show()
```



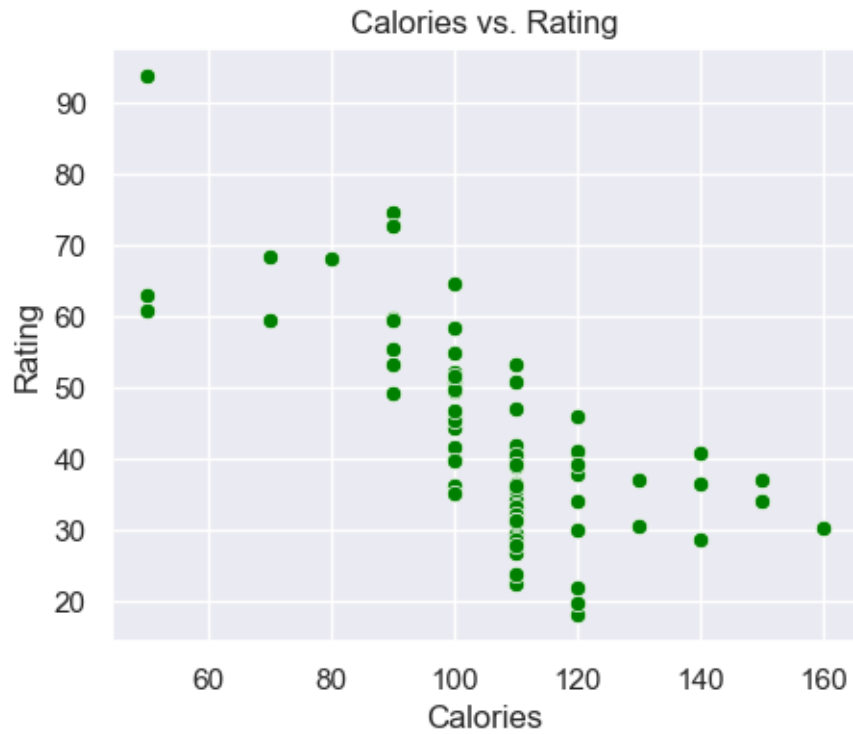
```
[16]: # Histogram
plt.figure(figsize=(5,4))
plt.hist(df['calories'], bins=10, color='salmon')
plt.title('Distribution of Calories per Serving')
plt.xlabel('Calories')
plt.ylabel('Frequency')
plt.show()
```



```
[20]: # Box plot
plt.figure(figsize=(6, 5))
sns.boxplot(x='sugars', y='mfr', data=df)
plt.xlabel('Manufacturer')
plt.ylabel('Grams of Sugar')
plt.title('Sugar Content by Manufacturer')
plt.show()
```



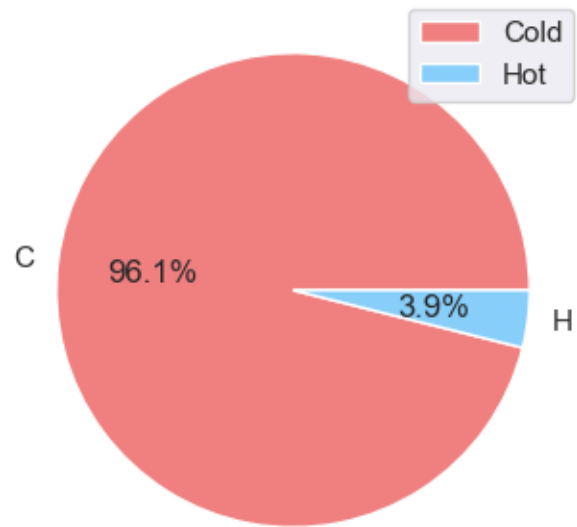
```
[21]: # Scatter plot
plt.figure(figsize=(5, 4))
sns.scatterplot(x='calories', y='rating', data=df, color='green')
plt.xlabel('Calories')
plt.ylabel('Rating')
plt.title('Calories vs. Rating')
plt.show()
```



```
[25]: # Pie plot
type_counts = df['type'].value_counts()
plt.figure(figsize=(5, 4))
plt.pie(type_counts, labels=type_counts.index, autopct='%1.1f%%',
        colors=['lightcoral', 'lightskyblue'])
plt.title('Proportion of Hot and Cold Cereals')
plt.ylabel('')
plt.legend(labels=['Cold', 'Hot'])

plt.show()
```

Proportion of Hot and Cold Cereals



[]: